

EXAMEN LA DISCIPLINA "PROGRAMAREA ALGORITMILOR" – SESIUNEA IANUARIE 2026 –

RESTRICȚII ȘI PRECIZĂRI:

1. Toate subiectele se vor rezolva folosind limbajul Python.
2. Rezolvările vor fi implementate exclusiv folosind facilitățile de bază ale limbajului Python, fără a utiliza module externe.
3. Subiectul 1 este obligatoriu, iar dintre subiectele 2, 3 și 4 se vor rezolva CEL MULT DOUĂ, la alegere.
4. Citirea datelor de intrare se va realiza de la tastatură, iar rezultatele vor fi afișate pe ecran.
5. Se garantează faptul că datele de intrare sunt corecte.
6. Operațiile de sortare se vor efectua folosind funcții sau metode predefinite din limbajul Python.
7. Rezolvările subiectelor alese dintre subiectele 2, 3 și 4 trebuie să conțină:
 - scurtă descriere a algoritmului și o argumentare a faptului că acesta se încadrează într-o anumită tehnică de programare;
 - în cazul problemelor rezolvate folosind metoda Greedy sau metoda programării dinamice se va argumenta corectitudinea criteriului de selecție sau a relațiilor de calcul;
 - în cazul problemelor rezolvate folosind metoda Backtracking se va argumenta relevanța condițiilor de continuare;
 - în cazul subiectelor unde se precizează complexitatea maximă pe care trebuie să o aibă soluția, se va argumenta complexitatea soluției propuse și vor primi punctaj maxim doar soluțiile corecte care se încadrează în complexitatea cerută;
 - în fiecare program Python se va preciza, pe scurt, sub forma unor comentarii, semnificația variabilelor utilizate.
8. Pentru subiectul 1 nu contează complexitatea soluțiilor propuse.
9. Rezolvările corecte care nu respectă restricțiile indicate vor primi punctaje parțiale.
10. Se acordă 1 punct din oficiu.

Subiectul 1 (3 p.)

a) Scrieți o funcție **nr_max_div** care primește ca parametru un număr variabil de numere naturale nenule și returnează o listă formată din toate numerele distincte, sortate crescător, dintre cele primite ca parametri, cu proprietatea că au un număr maxim de divizori proprii. De exemplu, pentru apelul **nr_max_div(1700, 1002, 288, 2026, 101, 980, 999, 288, 980)** funcția trebuie să furnizeze lista **[1700, 288, 980]** deoarece aceste numere au un număr maxim de divizori proprii, respectiv **16**. (1.5 p.)

b) Știind că **ldc** este o listă de șiruri de caractere, înlocuiți punctele de suspensie din instrucțiunea **ldc = [...]** cu o secvență de inițializare (*list comprehension*) astfel încât, după executarea sa, lista **ldc** să conțină toate cuvintele din lista de șiruri de caractere **cuvinte** care sunt formate din litere distincte. De exemplu, pentru **cuvinte = ['exemplul', 'usor', 'este', 'lista', 'vida']**, lista **ldc** va fi **['usor', 'lista', 'vida']** (0.5 p.)

c) Considerăm următoarea funcție recursivă:

```
def f(s):  
    if len(s) <= 1:  
        return s  
    else:  
        m = len(s) // 2  
        t = f(s[:m])  
        v = f(s[m:])  
        return t.replace(v[0], "") + v.replace(t[0], "")
```

Determinați complexitatea maximă (în cel mai defavorabil caz) a funcției apelată pentru un șir de caractere **s** de lungime **n** astfel: **f(s)**. (1 p.)

Subiectul 2 – metoda Greedy (3 p.)

Complexitatea maximă a soluției: $O(n \cdot \log(n))$

Pe un desen sunt n cercuri care au centrele pe dreapta Ox , numerotate $1, \dots, n$ ($1 \leq n \leq 100000$). Pentru fiecare cerc $i \in \{1, 2, \dots, n\}$ se cunosc coordonata x_i a centrului pe axa Ox și raza r_i . Să se determine un număr minim de cercuri care trebuie șterse de pe desen astfel încât cercurile rămase să fie exterioare două câte două (intersecția discurilor asociate cercurilor să fie vidă). Pentru aceasta, scrieți un program Python care citește de la tastatură numărul natural n (reprezentând numărul de cercuri), apoi pentru fiecare cerc i coordonata x_i și raza r_i (date pe o linie și separate printr-un spațiu, ca în exemplul de mai jos) și afișează indicii cercurilor care trebuie șterse.

Exemplu:

Date de intrare	Date de ieșire
4	2
1 1	4
2.5 1	
5 2	
7 8	

Observație: Soluția nu este neapărat unică!

Subiectul 3 – metoda Programării Dinamice (3 p.)

Complexitatea maximă a soluției: $O(n^2)$

Anstein se pregătește pentru un examen de matematică și are o **listă cu n probleme**, numerotate de la 1 la n . Când rezolvă o problemă i (evident, Anstein întotdeauna rezolvă o problemă corect!), el primește punctajul p_i corespunzător problemei, dar obosește și nu mai poate rezolva următoarele x_i probleme. Evident, Anstein vrea să **obțină cât mai multe puncte la finalul examenului**, de aceea vă roagă pe voi să scrieți un program Python care citește numărul natural n (dat pe o linie), apoi pentru fiecare problemă $i \in \{1, 2, \dots, n\}$ citește p_i și x_i (date pe o linie și separate printr-un spațiu, ca în exemplu) și afișează ce probleme trebuie să rezolve Anstein pentru a obține punctajul maxim posibil fără a se extenua (deci respectând x_i).

Exemplu:

Date de intrare	Date de ieșire
4	1 4
3 2	5
4 3	
4 4	
2 5	

Explicații: După ce rezolvă problema 1, Anstein primește 3 puncte, dar nu va putea rezolva următoarele $x_1 = 2$ probleme (adică problemele 2 și 3). După aceea, Anstein va rezolva problema 4 și primește 2 puncte, deci va obține un total de 5 puncte.

Subiectul 4 – metoda Backtracking (3 p.)

a) Loteria Română a extins jocul *Loto 6/49* la *Loto 6/n*, unde n este un număr natural cuprins între 13 și 100. Jucătorii împătimiți, care au observat cu atenție extragerile efectuate de-a lungul mai multor ani, au denumit *extrageri norocoase* acele extrageri formate din 6 numere care nu conțin numere consecutive, iar numerele nu au toate aceeași paritate. Scrieți un program Python care să citească de la tastatură numărul natural n , după care să afișeze toate extragerile norocoase distincte. **(2.5 p.)**

Exemplu:

Pentru $n = 14$ există 70 de extrageri norocoase:

1, 3, 5, 7, 9, 12	1, 3, 6, 8, 10, 14	<u>1, 4, 6, 9, 11, 13</u>	2, 4, 6, 9, 11, 14
1, 3, 5, 7, 9, 14	<u>1, 3, 6, 8, 11, 13</u>	1, 4, 6, 9, 11, 14	2, 4, 6, 9, 12, 14
1, 3, 5, 7, 10, 12	1, 3, 6, 8, 11, 14	1, 4, 6, 9, 12, 14	<u>2, 4, 7, 9, 11, 13</u>
<u>1, 3, 5, 7, 10, 13</u>	1, 3, 6, 8, 12, 14	1, 4, 6, 10, 12, 14	2, 4, 7, 9, 11, 14
1, 3, 5, 7, 10, 14	<u>1, 3, 6, 9, 11, 13</u>	<u>1, 4, 7, 9, 11, 13</u>	2, 4, 7, 9, 12, 14
1, 3, 5, 7, 11, 14	1, 3, 6, 9, 11, 14	1, 4, 7, 9, 11, 14	2, 4, 7, 10, 12, 14
1, 3, 5, 7, 12, 14	1, 3, 6, 9, 12, 14	1, 4, 7, 9, 12, 14	<u>2, 5, 7, 9, 11, 13</u>
1, 3, 5, 8, 10, 12	1, 3, 6, 10, 12, 14	1, 4, 7, 10, 12, 14	2, 5, 7, 9, 11, 14
<u>1, 3, 5, 8, 10, 13</u>	1, 3, 7, 9, 11, 14	1, 4, 8, 10, 12, 14	2, 5, 7, 9, 12, 14
1, 3, 5, 8, 10, 14	1, 3, 7, 9, 12, 14	1, 5, 7, 9, 11, 14	2, 5, 7, 10, 12, 14
<u>1, 3, 5, 8, 11, 13</u>	1, 3, 7, 10, 12, 14	1, 5, 7, 9, 12, 14	2, 5, 8, 10, 12, 14
1, 3, 5, 8, 11, 14	1, 3, 8, 10, 12, 14	1, 5, 7, 10, 12, 14	3, 5, 7, 9, 11, 14
1, 3, 5, 8, 12, 14	1, 4, 6, 8, 10, 12	1, 5, 8, 10, 12, 14	3, 5, 7, 9, 12, 14
1, 3, 5, 9, 11, 14	<u>1, 4, 6, 8, 10, 13</u>	1, 6, 8, 10, 12, 14	3, 5, 7, 10, 12, 14
1, 3, 5, 9, 12, 14	1, 4, 6, 8, 10, 14	<u>2, 4, 6, 8, 10, 13</u>	3, 5, 8, 10, 12, 14
1, 3, 5, 10, 12, 14	<u>1, 4, 6, 8, 11, 13</u>	<u>2, 4, 6, 8, 11, 13</u>	3, 6, 8, 10, 12, 14
1, 3, 6, 8, 10, 12	1, 4, 6, 8, 11, 14	2, 4, 6, 8, 11, 14	
<u>1, 3, 6, 8, 10, 13</u>	1, 4, 6, 8, 12, 14	<u>2, 4, 6, 9, 11, 13</u>	

b) Precizați cum ar trebui modificată o singură instrucțiune din program astfel încât să fie afișate doar *extragerile incerte*, adică extragerile norocoase care conțin numărul 13. Pentru exemplul anterior, aceste soluții sunt cele subliniate. **(0.5 p.)**